

Mi Primera Presentación

Un template básico de Beamer

Tu Nombre

Tu Institución

22 de marzo de 2026

- 1 Motivación
- 2 Generación del Dataset Simulado
- 3 Normalización de las labels
- 4 Pipeline de Entrenamiento

El problema: Medir la geometría de los poros

Material: AAO

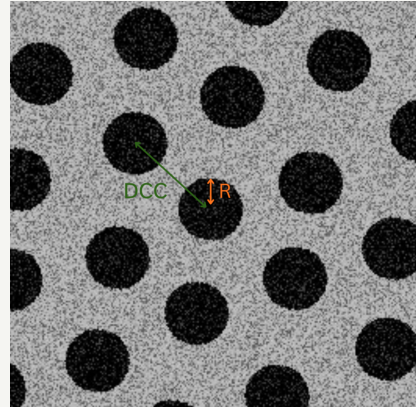
Membrana nanoestructurada con poros cilíndricos organizados en red hexagonal. Sus propiedades ópticas y mecánicas dependen de la geometría de los poros.

Objetivo

Predecir automáticamente los parámetros geométricos de una imagen SEM.

5 variables objetivo

R, σ_R	Radio promedio y dispersión [px]
DCC, σ_{DCC}	Distancia centro a centro [px]
P	Porosidad (%)

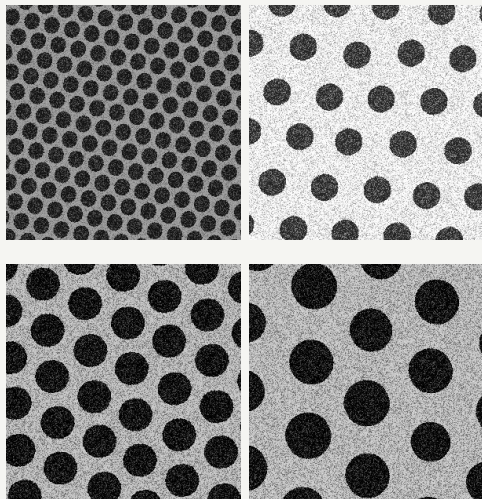


Generación del Dataset Simulado

Parámetros aleatorios por imagen

Parámetro	Rango
Radio r	$[10, 30]$ px
Distancia dcc	$[2,5r, 4,5r]$
Desorden en r, dcc	hasta 10 %
Rotación de red	$[0^\circ, 360^\circ]$
Brillo B	$[-40, 40]$
Contraste C	$[0,6, 1,4]$
Ruido	nivel 1

Se generaron **20 000 imágenes** de 300×300 px. Cada imagen genera una fila en el CSV con variables: $R, \sigma_R, DCC, \sigma_{DCC}, P$.



Normalización de las Labels

Por qué normalizar

Las variables tienen escalas muy distintas: $R \sim 20$ px pero $P \sim 0,3$. Sin normalizar, las variables grandes dominan el MAE y el modelo aprende mal.

StandardScaler

Técnica que transforma las variables:

$$x_{\text{norm}} = \frac{x - \mu}{\sigma}$$

donde μ y σ son la media y la desviación estándar de cada variable, respectivamente.

Resultado: media 0 y desv. est. 1.

Transformación Inversa

Despejar x de la ecuación:

$$x = x_{\text{norm}} \cdot \sigma + \mu$$

Importante: Los valores de μ y σ se guardan.

Flujo del proceso

- 1 CSV Original con 5 variables
- 2 Aplicar StandardScaler
- 3 Guardar scaler
- 4 Labels listas para entrenar
- 5 Entrenar la red neuronal
- 6 Evaluar la red neuronal y usar Scaler Inverso para obtener MAPE

Transformación de Imágenes

Valores originales

Las imágenes SEM vienen con píxeles en rango $[0, 255]$.

Normalización de píxeles

Se transforman a uno de estos rangos:

- Rango $[0, 1]$: dividir por 255
- Rango $[-1, 1]$: dividir por 127.5 y restar 1
- Mantener $[0, 255]$: sin transformación

¿Por qué?

Facilita convergencia del modelo y evita dominancia numérica de valores grandes.

División del Dataset

Partición

Las 20 000 imágenes se dividen en:

- **Train Set:** entrena el modelo
- **Validation Set:** valida en cada época
- **Test Set:** evalúa al final

Batching

Las imágenes se agrupan en lotes para:

- Optimizar memoria GPU/CPU
- Acelerar entrenamiento

Entrenamiento del Modelo

Arquitectura

Dos opciones:

- **CNN Completa:** Arquitectura de una red convolucional entrenada desde cero
- **Transfer Learning:** Se utilizan capas de un modelo preentrenado y se le agregan capas convolucionales al final que son entrenadas acordes a lo que se necesita predecir.

Datos de entrada

- Imágenes: transformadas a $[0, 1]$, $[-1, 1]$ o $[0, 255]$
- Labels: normalizados con StandardScaler

Proceso

Minimizar pérdida en Train Set validando en cada época.

Obtención del MAPE en Test Set

Paso 1: Predicciones

El modelo entrenado realiza predicciones en Test Set:

$$\text{Modelo}(x_{\text{img}}) = (\hat{R}_{\text{norm}}, \hat{\sigma}_{R,\text{norm}}, \hat{DCC}_{\text{norm}}, \hat{\sigma}_{DCC,\text{norm}}, \hat{P}_{\text{norm}})$$

Paso 2: Scaler Inverso

El modelo entrega predicciones en el espacio normalizado por lo que lo desnormalizamos:

$$(\hat{R}, \hat{\sigma}_R, \hat{DCC}, \hat{\sigma}_{DCC}, \hat{P}) = \text{scaler.inverse_transform}(\hat{R}_{\text{norm}}, \hat{\sigma}_{R,\text{norm}}, \dots)$$

Paso 3: Calcular MAPE

Calculamos MAPE para cada una de las variables en el Test Set:

$$\text{MAPE} = \frac{1}{n} \sum_{i=1}^n \left| \frac{y_i - \hat{y}_i}{y_i} \right| \times 100 \%$$

donde y_i son los labels reales del Test Set y $\hat{y}_i$ las predicciones.

Evaluación en Dataset Externo Real

Dataset externo

Se utilizaron **24 imágenes SEM reales** (no simuladas) con sus parámetros geométricos medidos manualmente para evaluar la generalización del modelo.

Objetivo

Verificar si el modelo entrenado con imágenes simuladas es capaz de predecir correctamente los parámetros de imágenes reales.

Métrica

MAPE sobre R , DCC y P en espacio real (unidades originales).

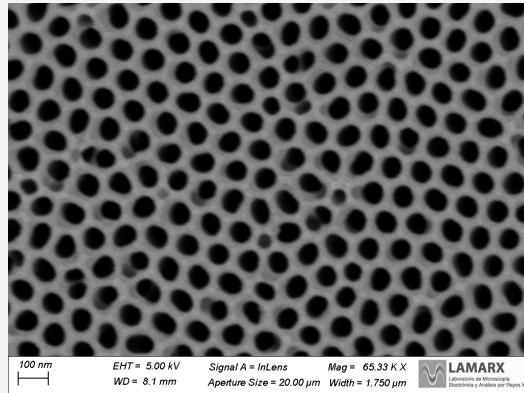


Imagen SEM real con banda de metadatos

Evaluación en Dataset Externo Real: Pipeline

Desafíos de las imágenes SEM reales

- 1 Las imágenes tienen resolución mas grande a la utilizada en el entrenamiento.
- 2 Las imágenes suelen incluir una franja impresa con parámetros del microscopio (escala, voltaje, etc.).

Solución: crop por esquina

El CSV incluye una columna `crop_corner` por imagen que indica desde qué esquina recortar (`tl/tr/bl/br/center`), permitiendo elegir manualmente una región de 300×300 px libre de la banda con parámetros.

Pipeline de evaluación

- 1 Leer labels en espacio real (sin estandarizar)
- 2 Recortar cada imagen a 300×300 px sin franja de parámetros
- 3 Normalizar píxeles igual que en el entrenamiento
- 4 Obtener predicciones y convertirlas al espacio real
- 5 Calcular MAPE sobre R , DCC , P

Nota

Las labels del CSV están en espacio real: el scaler solo se aplica a las predicciones para invertirlas al mismo espacio.

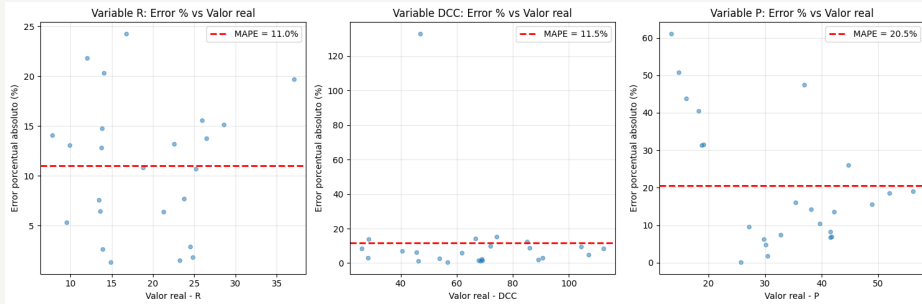
Evaluación en Dataset Externo Real: Resultados

MAPE — 24 imágenes SEM

Variable	MAPE
Radio R	11,00 %
Distancia DCC	11,53 %
Porosidad P	20,51 %

Conclusión

El modelo es aceptable: la mayoría de las imágenes presentan errores por debajo del 20 %.



¡Gracias!